# BLE and Python:

## How to build a simple BLE project on Linux with Python

PyConDE 2023 - Bruno Vollmer

biped

# Who am I ?

## Bruno Vollmer

CTO biped

RWTH AACHEN UNIVERSITY @ MSc CS

@brunovollmer5

https://brunovollmer.com

# Disclaimer

You can find the slides and the demo repository in the description of this talk.

I’m not an expert BLE or part of Bluez or the SIG.

# Outline

1. What is BLE?
2. Code Examples
3. Practical Tips and Common Pitfalls

**Not about:**

- In depth code examples
- BLE protocol in detail
- Communication optimisation

# Our use case

# biped, autonomous driving for humans

**bone** conduction headphones

computer

cameras

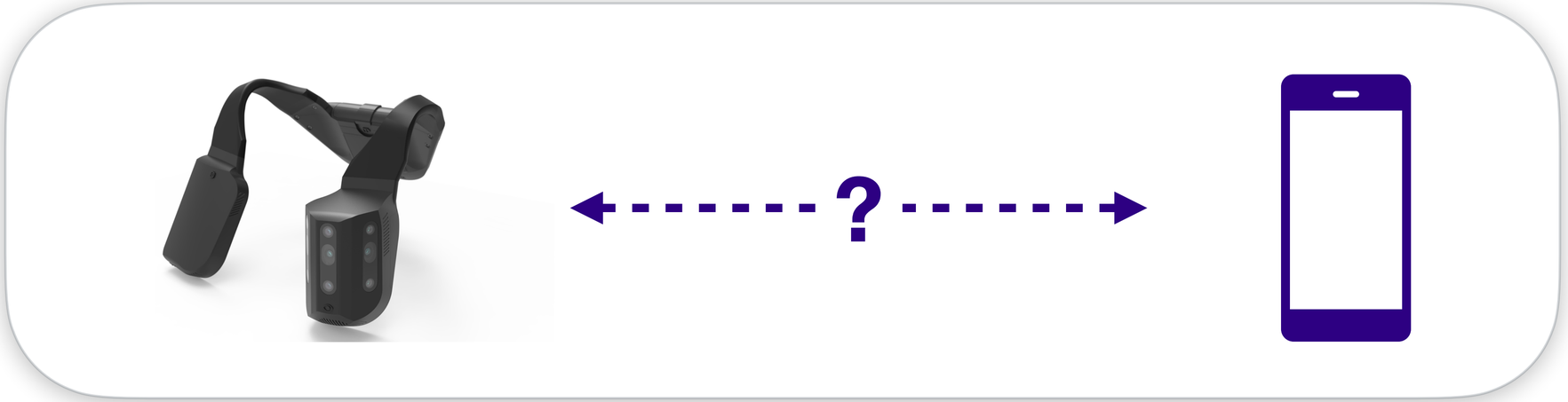
**sound** feedback like a car parking assist « bip »

**features:**

- detect all obstacles
- GPS instructions

using **AI**, filter based on risks

# Problem?




**Requirements:**

- Simple implementation
- Energy efficient
- Fast

Bluetooth®

Bluetooth® Low Energy

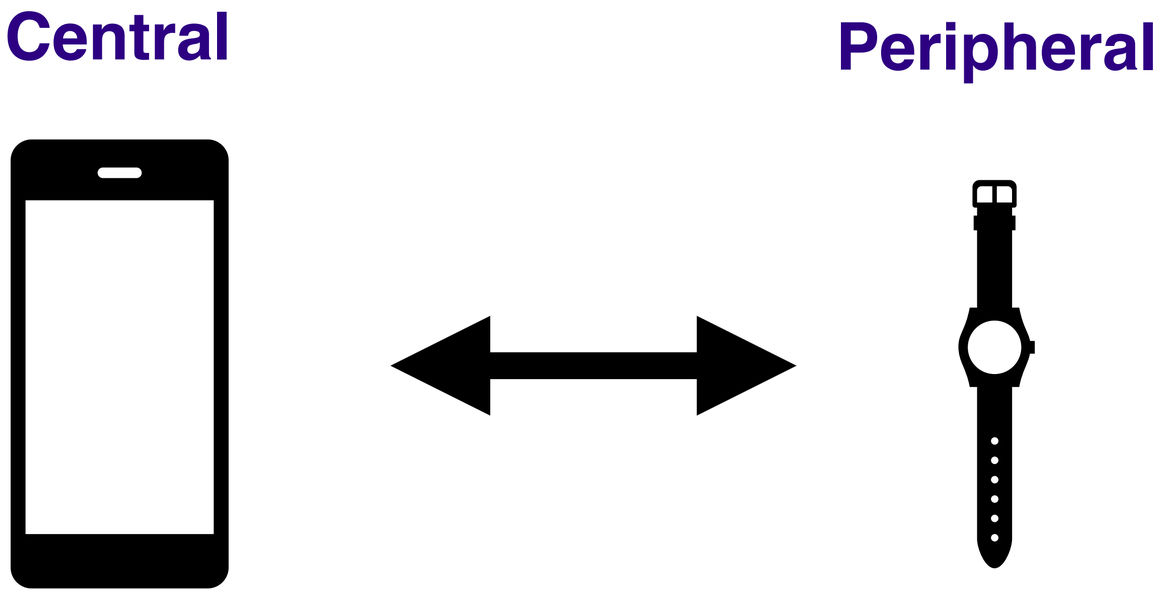
# How does BLE work?

# Bluetooth Low Energy

- Light-weight subset of classic Bluetooth
- Simple communication between devices and modern mobile platforms
- Devices stay “asleep" in between connections
- BLE connections are generally a lot faster then “classic” Bluetooth connections

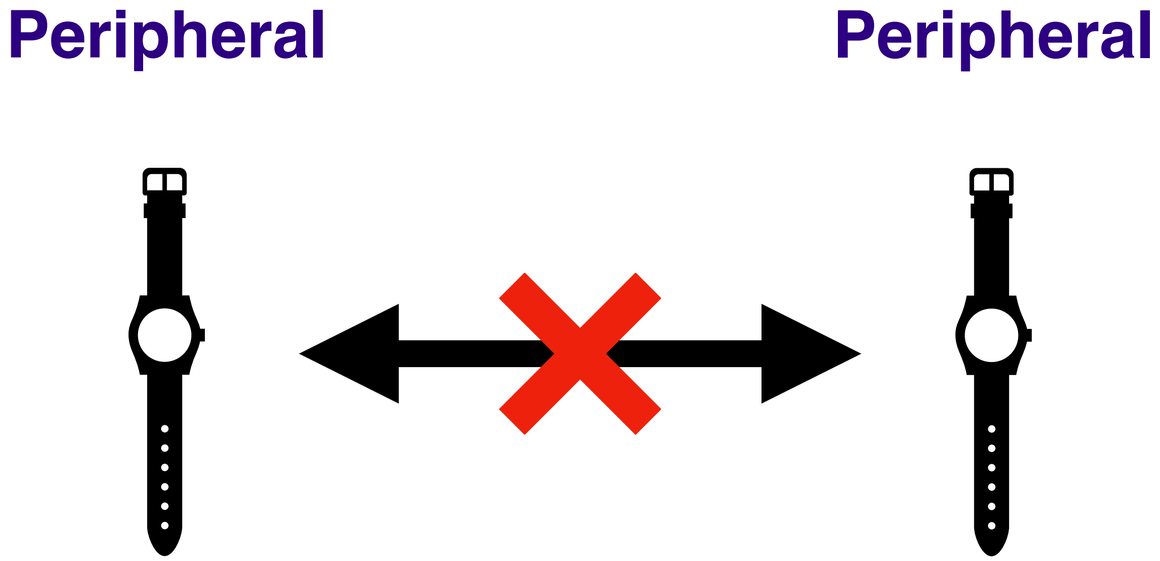

# Generic Access Profile

- **Generic Access Profile (GAP)** controls connections and advertising in Bluetooth
- Two types of devices
  - Peripheral
  - Central



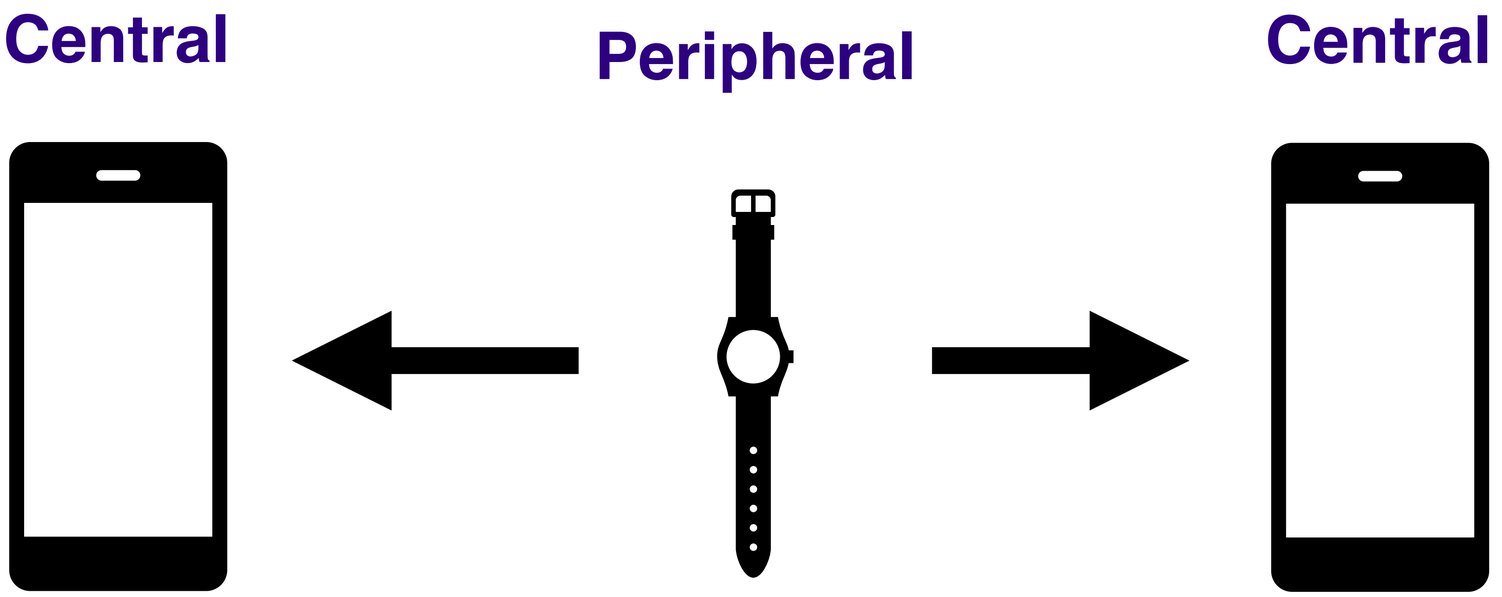
# Generic Access Profile

- **Generic Access Profile (GAP)** controls connections and advertising in Bluetooth
- Two types of devices
    - Peripheral
    - Central



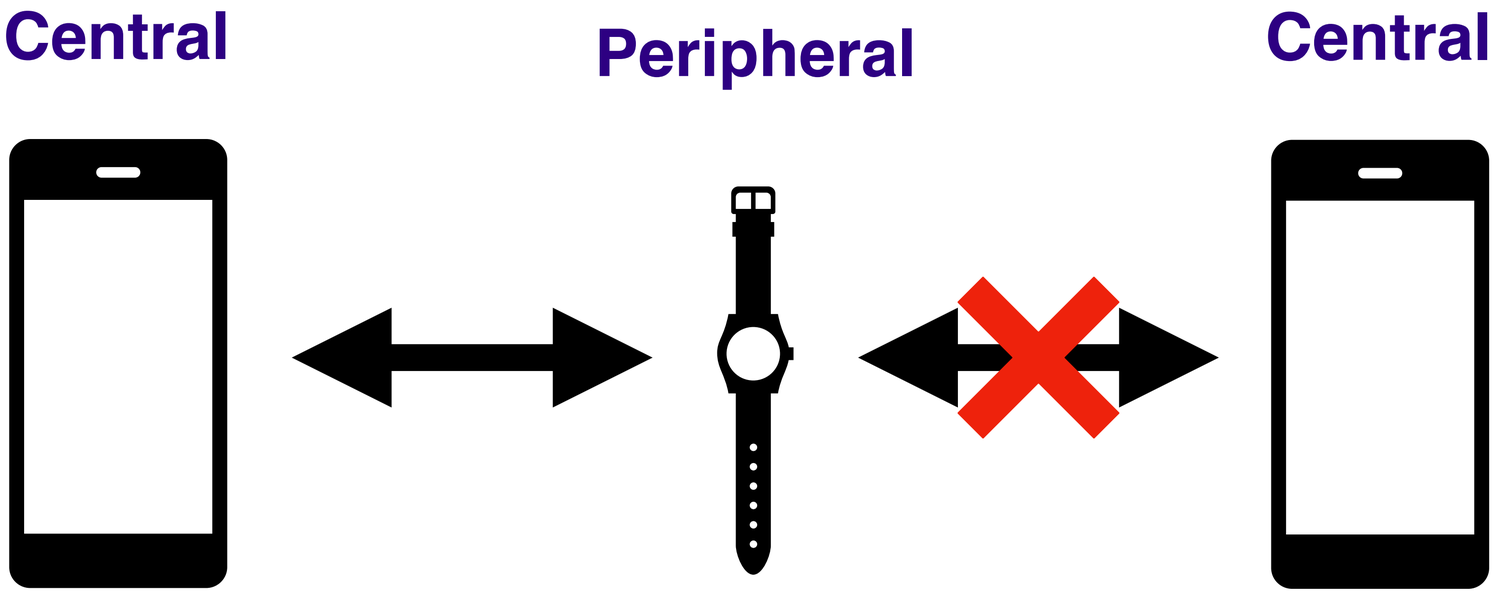
# Generic Access Profile

- **Generic Access Profile (GAP)** controls connections and advertising in Bluetooth
- Two types of devices
  - Peripheral
  - Central



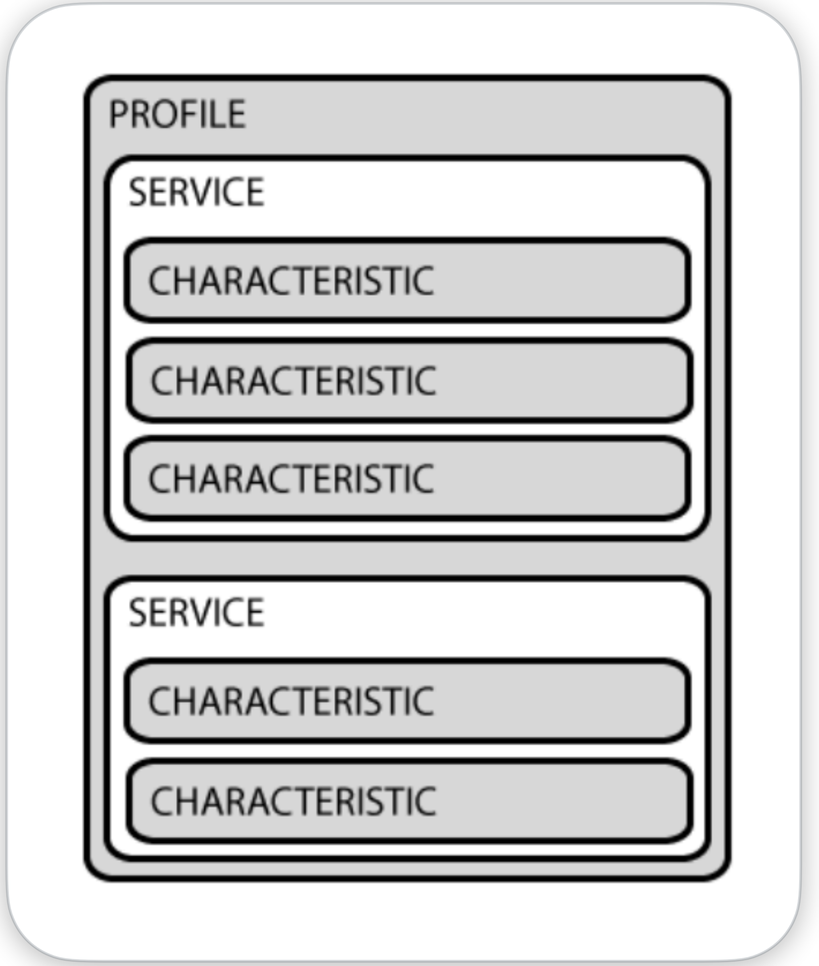
# Generic Attribute Profile

- **Generic ATTribute Profile (GATT)** defines the way that two devices transfer data back and forth
- Uses generic data protocol called the **Attribute Protocol (ATT)**



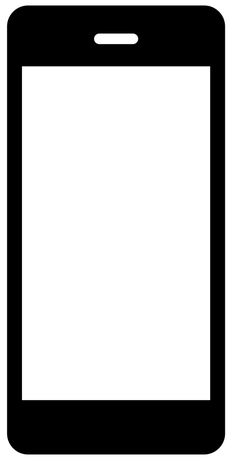
# Attribute Protocol

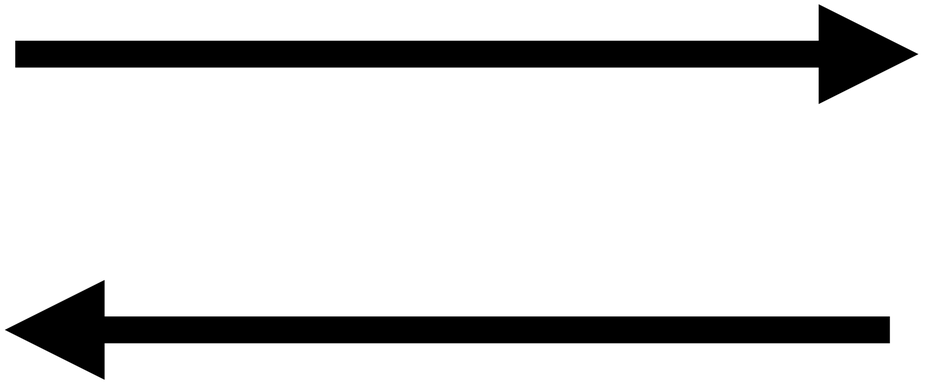
- Services are used to break data up into logical entities
- Characteristic encapsulate a single data point
- UUIDs can either be 16-bit (for officially adopted BLE Services/Characteristics) or 128-bit



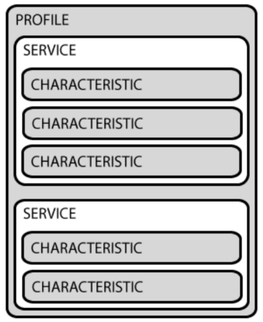
# Attribute Protocol

**GATT Client**



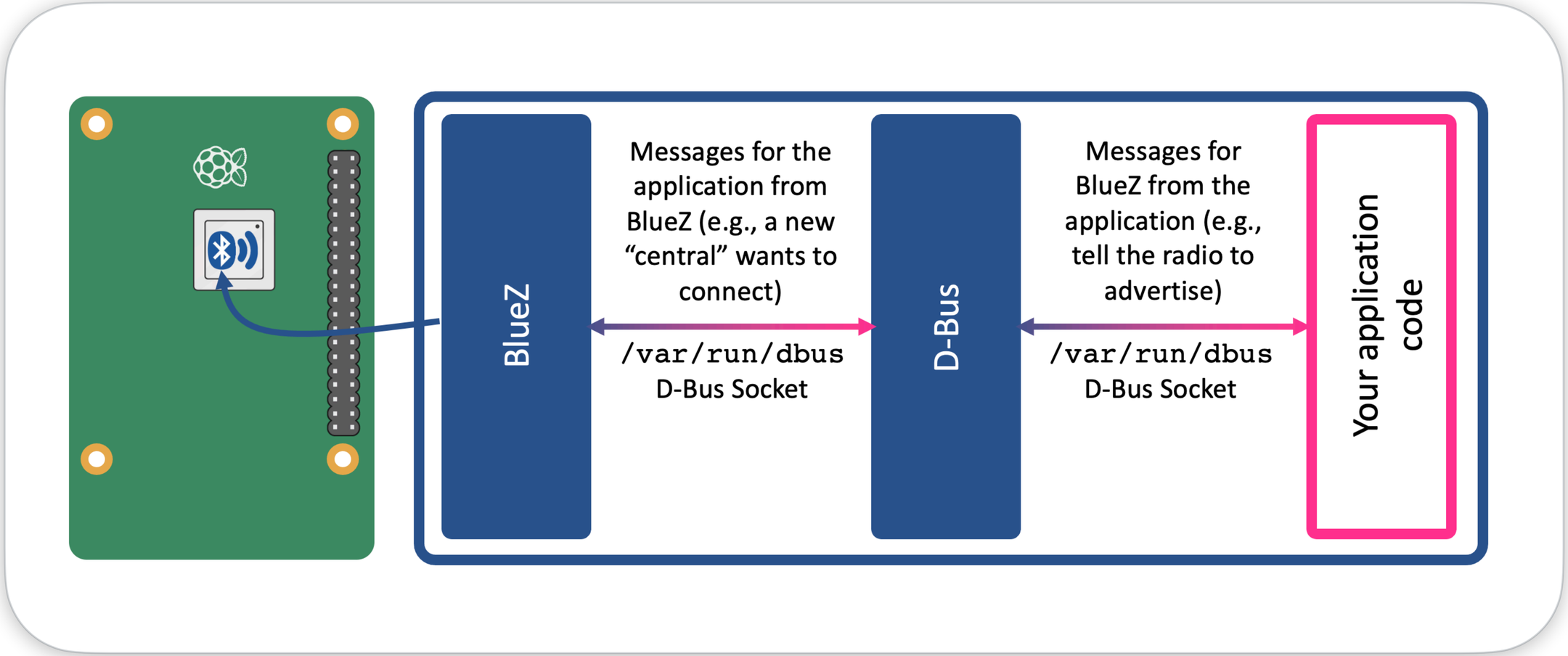


**GATT Server**



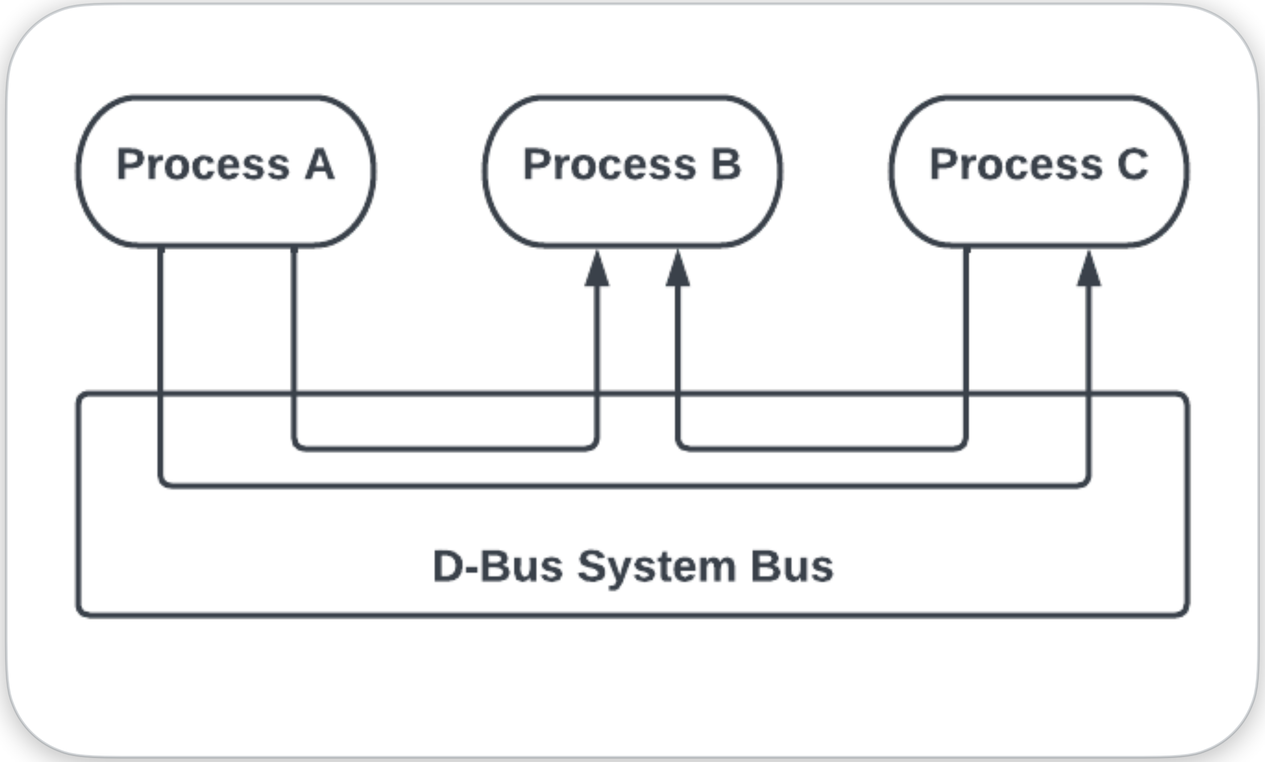
# BLE on Linux with Python

# BLE on Linux


BlueZ
Messages for the application from BlueZ (e.g., a new “central” wants to connect)
/var/run/dbus
D-Bus Socket
D-Bus
Messages for BlueZ from the application (e.g., tell the radio to advertise)
/var/run/dbus
D-Bus Socket
Your application code

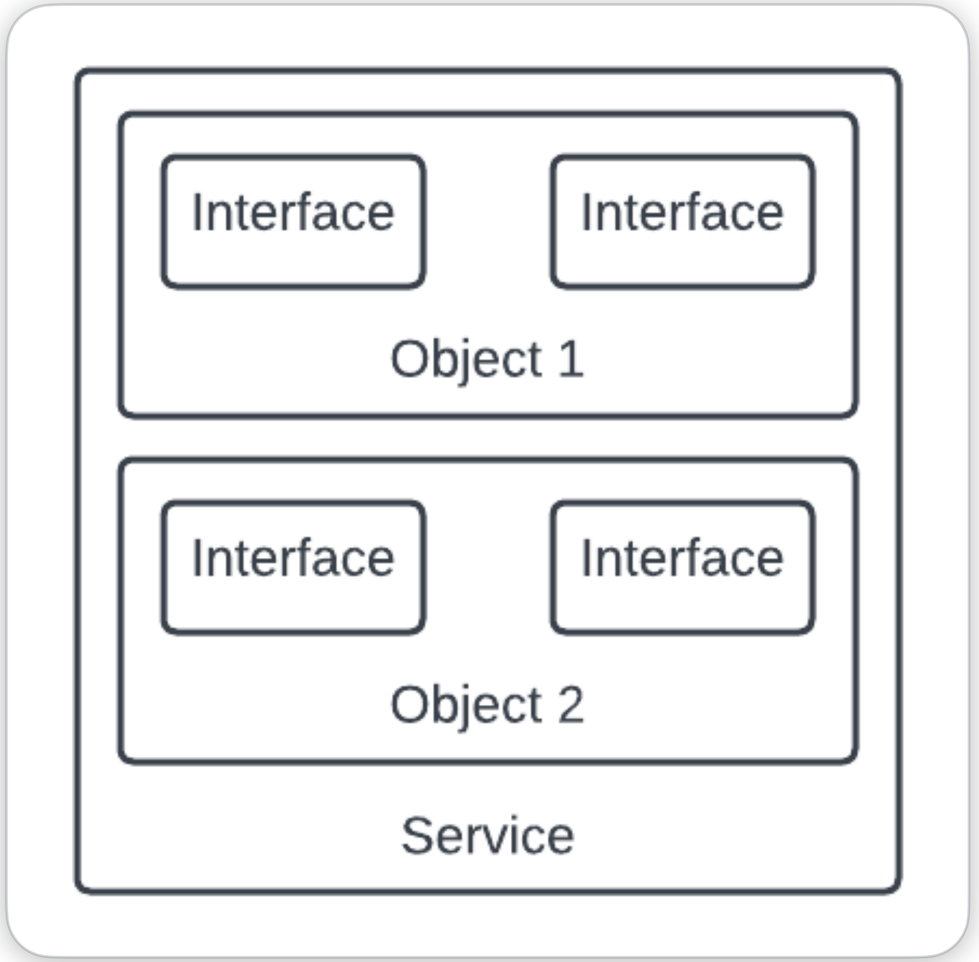
# D-BUS

- D-Bus is a message-oriented middleware mechanism
- Allows communication between multiple processes



# D-BUS

- D-Bus is a message-oriented middleware mechanism
- Allows communication between multiple processes
- Important concepts within D-Bus
  - Service
  - Object
  - Interface



# D-BUS Example

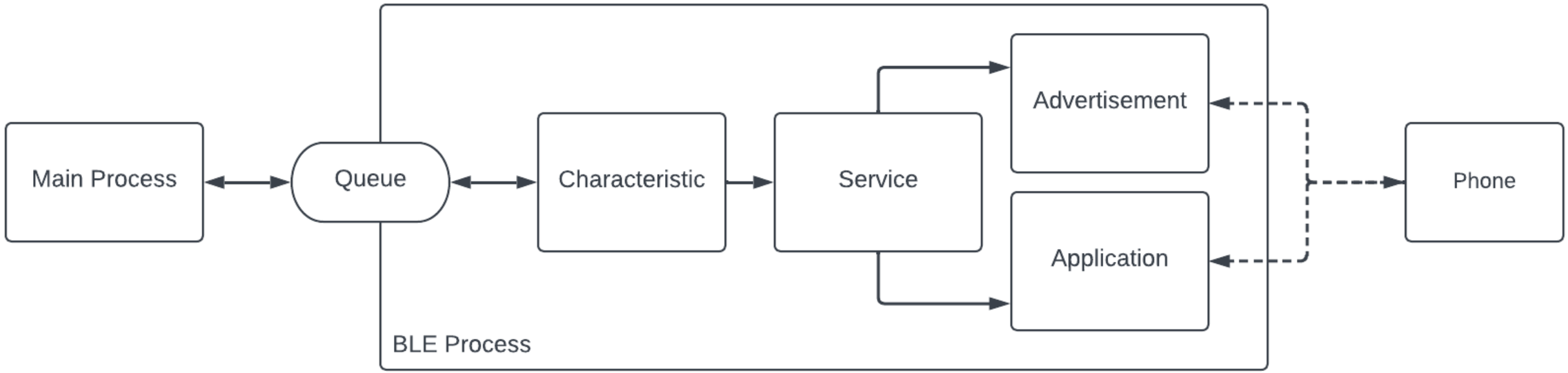
```
class Descriptor(dbus.service.Object):
    """
    org.bluez.GattDescriptor1 interface implementation
    """

    def __init__(self, bus):
        self.path = characteristic.path + "/desc"
        self.bus = bus
        dbus.service.Object.__init__(self, bus, self.path)

    @dbus.service.method(DBUS_PROP_IFACE, in_signature="s", out_signature="a{sv}")
    def GetAll(self, interface):
        ...
```

# Code Demo

# Code Example Architecture



# Tipps & Tricks

# Tools BLE

**bluetoothctl**

- Lets us interact with bluez through the terminal
- Manually create GATT server

```
$ bruno@linux ~ bluetoothctl
Agent registered
[CHG] Controller 10:2C:6B:CB:DA:EE Pairable: yes
[bluetooth]# power on
Changing power on succeeded
```

# Debugging BLE

```
$ bruno@linux ~ sudo dbus-monitor --system "destination='org.bluez'" "sender='org.bluez'"
...
method call time=1681483900.289489 sender=:1.40 -> destination=org.bluez serial=41
path=/org/bluez/hci0; interface=org.freedesktop.DBus.Properties; member=Set
   string "org.bluez.Adapter1"
   string "Powered"
   variant       boolean true
method return time=1681483900.290347 sender=:1.1 -> destination=:1.40 serial=55
reply_serial=41
```

# Tools BLE

**btmon**

- Bluetooth subsystem monitor infrastructure for reading HCI traces
- Very extensive output

```
> HCI Event: Extended Inquiry Result (0x2f) plen 255                    #36 [hci0]
22.235820
        Num responses: 1
        Address: 54:2A:A2:ED:5E:27 (Alpha Networks Inc.)
        Page scan repetition mode: R1 (0x01)
        Page period mode: P0 (0x00)
        Class: 0x28043c
          Major class: Audio/Video (headset, speaker, stereo, video, vcr)
          Minor class: Video Display and Loudspeaker
          Capturing (Scanner, Microphone)
          Audio (Speaker, Microphone, Headset)
        Clock offset: 0x4a73
        RSSI: -93 dBm (0xa3)
@ MGMT Event: Device Found (0x0012) plen 19                          {0x0001} [hci0]
22.235858
        BR/EDR Address: 54:2A:A2:ED:5E:27 (Alpha Networks Inc.)
        RSSI: -93 dBm (0xa3)
        Flags: 0x00000001
          Confirm Name
        Data length: 5
        Class: 0x28043c
          Major class: Audio/Video (headset, speaker, stereo, video, vcr)
          Minor class: Video Display and Loudspeaker
          Capturing (Scanner, Microphone)
          Audio (Speaker, Microphone, Headset)
@ MGMT Command: Confirm Name (0x0025) plen 8                         {0x0001} [hci0]
22.236284
        BR/EDR Address: 54:2A:A2:ED:5E:27 (Alpha Networks Inc.)
        Name known: No (0x00)
```

# Debugging BLE

**Possible problems**

- BLE advertisement is not showing on external device

  ➡ Monitor connection of application with Bluez via DBUS monitoring

- Communication between external server and client has a bug

  ➡ Write and read test values with external test app

- Communication is unstable or certain messages don’t arrive

  ➡ Monitor packets with btmon to find possible cause

# Extra information and Pitfalls

**Common pitfalls:**

- Be aware of max byte sizes defined by ATT

  ➡ Not possible to advertise two custom services at once

- iOS and Android have different BLE implementations

  ➡ Characteristics have different max data size

  ➡ No pairing before a connection might effect stability

- iOS requires a certain set of connection parameters to guarantee a robust connection

# Sources


- https://www.bluetooth.com/blog/the-bluetooth-for-linux-developers-study-guide/
- https://developer.apple.com/library/archive/qa/qa1931/_index.html
- https://learn.adafruit.com/introduction-to-bluetooth-low-energy
- https://litum.com/what-is-ble-how-does-ble-work/
- https://dbus.freedesktop.org/doc/dbus-python/
- https://punchthrough.com/creating-a-ble-peripheral-with-bluez/
- https://software-dl.ti.com/lprf/simplelink_cc2640r2_latest/docs/blestack/ble_user_guide/html/ble-stack-3.x/gap.html#connection-parameters
- https://www.raspberrypi-bluetooth.com/bluez-5_5-and-dbus.html

thank you
for listening

biped